CLAUDE CERTIFIED ARCHITECT - FOUNDATIONS | EXAM CODE CCAR-F

Final Study and Revision Guide

Companion to CCAF Mock Exams 01-05
Performance review, error analysis, and domain revision

Use this guide with: CCAF_Mock_Exam_01.pdf to CCAF_Mock_Exam_05.pdf (300 original questions in total).

Status: ORIGINAL PRACTICE MATERIAL. Not an official Anthropic document; it summarizes the public Exam Guide (CCAR-F v1.0, effective July 2026) and the practice papers.

Honest limits: a raw percentage on these papers is not a scaled score, and no practice material can guarantee a pass.

Contents

1. Purpose and honest limits 3

2. Verified official facts (Exam Guide v1.0) 3

3. Using the five papers as a progressive sequence 4

4. Score tracking worksheet 5

5. Practice results (all 300 items): design statistics 6

6. Classifying and diagnosing wrong answers 7

7. Domain revision notes 8

Domain 1 - Agentic Architecture & Orchestration (27%) . . . 8

Domain 2 - Tool Design & MCP Integration (18%) . . . 8

Domain 3 - Claude Code Configuration & Workflows (20%) . . . 9

Domain 4 - Prompt Engineering & Structured Output (20%) . . . 9

Domain 5 - Context Management & Reliability (15%) . . . 9

8. Pattern-recognition cheat sheet 11

9. Cross-reference: official task statements to practice questions 12

10. Final week plan and exam-day checklist 13

1. Purpose and honest limits

This guide turns five timed practice papers into a revision plan. It tells you how to read your results, how to separate real knowledge gaps from reading slips, and what to revise next. It also summarizes the official scope so you can check the papers against it.

• The papers are **original and unofficial**. They follow the public guide's format, domains, and scenarios, but their difficulty is not psychometrically calibrated and may differ from the real exam.
• The real exam reports a **scaled score from 100 to 1,000 with a cut score of 720**, set by a formal standard-setting study (criterion-referenced, not graded against other candidates). A raw percentage here does not convert to that scale. Domain percentages on the real score report are informational; the pass/fail decision uses the total scaled score.
• Practice results are signals about your preparation, not predictions. Preparing with these papers alone does not guarantee a pass; combine them with hands-on building, as the official guide recommends.

2. Verified official facts (Exam Guide v1.0)

Item	Official value
Credential / code	Claude Certified Architect - Foundations / CCAR-F
Items and format	60 items; multiple-choice and multiple-response; each item states how many responses to select
Structure	4 scenarios drawn from a bank of 6
Time limit	120 minutes (proctored; online or test center)
Passing score	Scaled 720 on a 100-1,000 scale; result is pass/fail plus scaled score and percent-correct by domain
Domain weights	1 Agentic Architecture & Orchestration 27%; 2 Tool Design & MCP Integration 18%; 3 Claude Code Configuration & Workflows 20%; 4 Prompt Engineering & Structured Output 20%; 5 Context Management & Reliability 15%
Explicitly out of scope	Fine-tuning; API authentication, billing, quotas, pricing; hosting MCP servers; computer use; vision; streaming; OAuth and key rotation; cloud-provider configuration; prompt caching internals; tokenization; model benchmarking
Validity and retakes	Credential valid 12 months; retake waits of 14, 30, and 90 days after failed attempts; up to four attempts per rolling 12 months

3. Using the five papers as a progressive sequence

Step	What to do	Aim
Paper 1 (diagnostic)	Sit it cold, timed, closed-book. Score by domain. Log every miss with an error type (section 6).	Find baseline strengths and the two weakest domains.
Revise	Use section 7 on your weakest domains and the cross-reference in section 9 to re-read the matching official task statements.	Close knowledge gaps before testing again.
Papers 2 and 3	Sit each at least two days after revising. Compare domain scores with Paper 1; expect different scenarios (the scenario mix rotates).	Check transfer to new contexts, not memory of answers.
Targeted repair	Rewrite, in your own words, the rule behind each miss. Re-answer missed items only after 48 hours.	Convert misses into rules you can apply.
Paper 4	Full simulation: one sitting, 120 minutes, no interruptions, mark-for-review allowed.	Pacing and stamina.
Paper 5 (readiness)	Sit it after a final revision pass. Treat consistent domain scores above about 75% across Papers 3-5 as a good sign, and any domain under about 65% as a priority.	Confirm readiness; plan last-week review.

The 65% and 75% figures are practice heuristics for these papers, not official cut points. Do not memorize answers: option order differs from any other source, and the real exam uses different questions.

Paper-by-scenario coverage

Scenario	P1	P2	P3	P4	P5	Total
1. Customer Support Resolution Agent	17	15	-	15	-	47
2. Code Generation with Claude Code	14	16	-	13	13	56
3. Multi-Agent Research System	16	-	15	-	16	47
4. Developer Productivity with Claude	13	-	17	15	-	45
5. Claude Code for Continuous Integration	-	14	15	-	14	43
6. Structured Data Extraction	-	15	13	17	17	62

4. Score tracking worksheet

Enter the number correct per domain after each paper (questions per domain are the same in every paper: 16, 11, 12, 12, 9).

Domain	Max	P1	P2	P3	P4	P5	Total / %
1. Agentic Architecture & Orchestration	16						
2. Tool Design & MCP Integration	11						
3. Claude Code Configuration & Workflows	12						
4. Prompt Engineering & Structured Output	12						
5. Context Management & Reliability	9						
Overall	**60**						

Reading the table

• **Strongest and weakest domains:** rank domains by percent correct, but weigh sample size. Domain 5 has only 9 questions per paper, so one question moves it by 11 points. Use totals across papers (45 questions in Domain 5, 80 in Domain 1) before drawing conclusions.
• **Trend, not a single paper:** look for a domain that stays low across two papers (a real gap) versus one that dips once (often a scenario effect).
• **Weight your effort:** Domain 1 is 27% of the real exam and Domain 5 is 15%; a gap in Domain 1 is usually the better use of limited time, all else equal.
• **Timing:** 120 minutes for 60 items is 2 minutes per item. Aim to finish a first pass in about 90 minutes and keep 25-30 minutes for flagged items and multiple-response checks.

5. Practice results (all 300 items): design statistics

The bank contains 300 questions: 270 single-answer, 25 select-two, and 5 select-three. Primary domains: D1=80; D2=55; D3=60; D4=60; D5=45. Difficulty labels (design intent): 61 foundational, 164 intermediate, 75 challenging.

Difficulty labels were assigned during design and checked by editorial review. They are not derived from candidate response data, so treat them as rough guides.

Domain by scenario (question counts across all five papers)

Domain	S1	S2	S3	S4	S5	S6	Total
1. Agentic Architecture & Orchestration	21	7	21	10	9	12	80
2. Tool Design & MCP Integration	9	5	13	9	6	13	55
3. Claude Code Configuration & Workflows	-	31	-	16	13	-	60
4. Prompt Engineering & Structured Output	5	5	6	6	14	24	60
5. Context Management & Reliability	12	8	7	4	1	13	45

S1 Customer Support; S2 Claude Code generation; S3 Multi-agent research; S4 Developer productivity; S5 CI; S6 Structured extraction.

6. Classifying and diagnosing wrong answers

For each wrong or guessed item, write one line in the log and assign exactly one primary error type.

Type	What it looks like	How to tell	Fix
K - knowledge gap	You could not state the rule, or the key's reasoning is new to you.	Read only the stem and answer in your own words, without options. If you cannot, it is a gap.	Re-read the official task statement; build or test the feature; write the rule in one sentence.
R - reading slip	You knew the concept but missed a qualifier (BEST, FIRST, SAFEST), the count in 'Select TWO', or a number in the stem.	You can explain the key immediately after seeing it, and you can find the missed phrase in the stem.	Underline qualifiers and counts before reading options; reread the stem last.
T - plausible-distractor trap	You chose an option that is useful in general but wrong under this stem (for example more prompting where a hook is needed).	You can name a context where your choice would be right, but the stem's constraint rules it out.	For each trap, write 'works when ..., fails here because ...'.
A - added assumption	You imported a fact not in the stem to justify an option.	Your reasoning contains 'probably' or 'maybe the system also ...'.	Use only the stated constraints; if you need an extra assumption, the option is likely a distractor.
G - guess / second-guess	You changed a right answer or chose without reasoning.	No written reason at the time.	Mark uncertain items; on review, change an answer only for a concrete new reason.

A quick test to separate a knowledge gap from a reading mistake: cover the options, restate what the stem is really asking in one sentence, and say what the best action would be. If your own answer matches the key, it was a reading or trap error. If it does not, it was a knowledge gap. Count each type per domain: many K entries mean study; many R or T entries mean practice on question technique.

Wrong-answer log (copy as needed)

Paper-Q	Domain / task	Type K/R/T/A/G	Rule in one sentence	Recheck date

7. Domain revision notes

Each row lists what to know and the classic trap seen in the practice items. Cross-check against the official Exam Guide, Section 6.

Domain 1 - Agentic Architecture & Orchestration (27%)

Task	Know this	Classic traps
1.1 Agentic loops	Continue on stop_reason 'tool_use' (run tools, append tool_result for every tool_use ID); stop on 'end_turn'; treat 'max_tokens' as truncation, not completion. The model chooses the next tool; code does not parse prose.	Terminating on text content or phrases; an iteration cap as the primary stop; sending only the newest message (the API is stateless).
1.2 Coordinator-subagent	Hub-and-spoke: the coordinator decomposes, delegates, aggregates, routes information, and handles errors. Select subagents by query complexity; partition scope; refine iteratively until coverage is sufficient.	Subagent-to-subagent calls; always running the full pipeline; blaming downstream agents when the decomposition was narrow.
1.3 Invocation and context	Task tool; allowedTools must include "Task"; subagents have isolated context and no shared memory; pass complete findings in the prompt, structured to separate content from metadata; parallel subagents via multiple Task calls in one response; prompts state goals and quality criteria.	Assuming inherited history; prompts that prescribe step-by-step procedures; spawning in separate turns for independent work.
1.4 Enforcement and handoff	Programmatic prerequisites and gates for financial or identity rules (prompts have a non-zero failure rate). Handoffs carry customer ID, root cause, amounts, recommended action because humans lack the transcript.	Few-shot or stronger wording as the fix for a rule that must always hold; handoffs that say 'customer is upset'.
1.5 Hooks	PostToolUse normalizes or trims results before the model sees them; pre-call hooks block or redirect (for example refunds over a threshold). Hooks verify facts themselves.	Gating on a flag the model sets; per-call checks that miss cumulative limits; prompt rules for hard business rules.
1.6 Decomposition	Prompt chaining for predictable multi-aspect work; adaptive decomposition for open-ended work; per-file local passes plus a cross-file integration pass.	A fixed checklist for an unknown cause; one giant pass over many files.
1.7 Sessions	--resume <name> continues a prior session when its context is mostly valid; tell it which files changed; start fresh with a structured summary when tool results are stale; fork_session branches from a shared baseline (keep branches draft-only if they can cause side effects).	Resuming into stale tool results; assuming a fork copies the repository.

Domain 2 - Tool Design & MCP Integration (18%)

Task	Know this	Classic traps
2.1 Tool interfaces	Descriptions drive selection: inputs, example queries, edge cases, boundaries versus similar tools. Rename or split overlapping tools. Check system prompts for keyword rules that override descriptions.	Adding more overlapping tools; keyword routing in prompts; one-word descriptions.
2.2 Error responses	isError with category (transient, validation, business, permission), retryability, and a human-readable description. Distinguish access failure from valid empty result. Local recovery first; propagate unresolved errors with context.	Generic 'Operation failed'; empty success on failure; retrying non-retryable errors.
2.3 Tool distribution and tool_choice	4-5 role-relevant tools per agent; scoped cross-role tools; constrained replacements (load_document for fetch_url). 'auto' may return text; 'any' guarantees a tool call; forced names a tool (use only for the step that must run first, then continue).	Giving every agent every tool; forcing a tool on every turn; relying on 'auto' for guaranteed structure.
2.4 MCP integration	Project .mcp.json for shared servers with ${ENV} expansion; ~/.claude.json for personal or experimental; all servers' tools available at once; resources expose catalogs; enrich descriptions so agents do not prefer built-ins; reuse community servers for standard integrations.	Committing secrets; assuming user-level servers are shared; exploration calls for stable catalogs.
2.5 Built-in tools	Grep for contents, Glob for paths, Read/Write for whole files, Edit for unique-anchor changes (Read + Write when no unique anchor). Explore incrementally: Grep, Read, follow imports; search every exported name or wrapper.	Glob for content; reading everything up front; global shell replacements.

Domain 3 - Claude Code Configuration & Workflows (20%)

Task	Know this	Classic traps
3.1 CLAUDE.md	User (~/.claude/CLAUDE.md, personal), project (shared via version control), directory-level; @import for modular standards; .claude/rules/ for topic files; /memory shows what is loaded.	Putting team rules in a user-level file; one giant file; copying standards instead of importing.
3.2 Commands and skills	.claude/commands/ (project, shared) versus ~/.claude/commands/ (personal); skills in .claude/skills/ with SKILL.md frontmatter: context: fork, allowed-tools, argument-hint. Skills are on-demand; CLAUDE.md is always loaded.	Confusing argument-hint with tool restriction; sharing user-level files; loading rare workflows every session.
3.3 Path-specific rules	.claude/rules/*.md with YAML paths globs load only when matching files are involved; glob rules beat directory CLAUDE.md for conventions spread across directories (for example **/*.test.*).	Single-level globs that miss nested folders; missing paths so rules load always.
3.4 Plan mode versus direct	Plan mode for large, multi-file, architectural, or multi-approach work (Explore subagent isolates discovery); direct execution for clear, well-scoped changes; combine them for migrations.	Planning trivial fixes; executing large restructurings without a design.
3.5 Iterative refinement	2-3 concrete input/output examples; tests first then iterate on failures; interview pattern in unfamiliar domains; one message for interacting issues, sequential fixes for independent ones.	Repeating prose with more emphasis; tests derived from the implementation.
3.6 CI/CD	-p / --print for non-interactive runs; --output-format json with --json-schema for machine-parseable findings; CLAUDE.md for review criteria and fixtures; independent review instance; prior findings in context to avoid duplicates; existing tests in context.	Interactive invocations in CI; scraping markdown; same-session self-review.

Domain 4 - Prompt Engineering & Structured Output (20%)

Task	Know this	Classic traps
4.1 Explicit criteria	Name what to report and what to skip; define severity levels with concrete examples; temporarily disable a high false-positive category to protect trust.	'Be conservative' or 'only high-confidence' as the fix.
4.2 Few-shot	2-4 targeted examples for ambiguous cases with reasoning; demonstrate format; contrast acceptable patterns with genuine issues; cover varied document structures.	Dozens of near-identical examples; examples that encode outdated style.
4.3 Structured output	tool_use with JSON schema eliminates syntax errors, not semantic errors; nullable fields prevent fabrication; enums with 'other' plus detail and 'unclear'; normalization rules alongside schemas; money as integer minor units or decimal strings plus currency.	Required fields for data that may be absent; trusting schema validity as correctness.
4.4 Validation and retry	Retry with the document, the failed output, and the specific error; retries cannot recover absent information; calculated_total versus stated_total; conflict_detected; detected_pattern to analyze dismissals; bound retries then route to review.	Blind retries; defaults that hide missing data.
4.5 Batch processing	Message Batches API: 50% savings, up to 24 hours, no latency guarantee, no multi-turn tool calling inside a request, custom_id correlation. Use for non-blocking work; compute worst-case SLA = submission wait + 24 h (plus retries); refine the prompt on a sample first; resubmit only failures.	Batching blocking checks; assuming result order; ignoring the retry path in SLAs.
4.6 Multi-instance review	A generator is biased toward its own work; use an independent instance without its reasoning; per-file plus integration passes; confidence per finding, calibrated before routing.	Same-session self-review; three-run consensus that suppresses intermittent true bugs.

Domain 5 - Context Management & Reliability (15%)

Task	Know this	Classic traps
5.1 Conversation context	Case-facts block for amounts, dates, order numbers, expectations; trim verbose tool output; key findings first with headers; upstream agents return structured facts; send full history on each request.	Progressive summaries of numbers; padding the middle of long inputs.

Task	Know this	Classic traps
5.2 Escalation	Escalate for explicit human requests (immediately), policy gaps, and inability to progress; resolve when frustrated but capable; ask for identifiers when several records match; clarify before irreversible actions.	Sentiment or self-reported confidence as triggers; guessing among matches.
5.3 Error propagation	Structured error context (type, attempted query, partial results, alternatives); local recovery for transient failures; coverage annotations; valid empty results are reportable findings.	Generic statuses; silent suppression; terminating everything on one failure.
5.4 Codebase exploration	Scratchpad files for findings; subagents isolate verbose exploration; summaries injected into the next phase; manifests for crash recovery; /compact when context fills.	Trusting a long context to retain specifics; re-running whole jobs after a crash.
5.5 Human review	Segment accuracy by document type and field; stratified sampling of high-confidence output; field-level confidence calibrated on labeled data; route ambiguous or contradictory sources to humans.	Relying on an aggregate number; uncalibrated thresholds.
5.6 Provenance	Claim-source mappings preserved through synthesis; annotate conflicts with attribution; require dates; render tables for financial data, prose for news, lists for technical findings.	Averaging conflicting values; dropping sources during summarization.

8. Pattern-recognition cheat sheet

If the stem says ...	Think ...
A rule must hold every time (money, identity, compliance)	Programmatic gate or hook, not a prompt. Check the real quantity (cumulative, ownership), and verify in code.
Tool is picked wrongly or interchangeably	First fix descriptions and boundaries; check the system prompt for keyword rules; then split, rename, or reduce the tool set.
Stale facts after time or code changes	Fresh session with a structured summary and re-fetched volatile data; if resuming, name the changed files.
Output is valid JSON but wrong	Semantic validation: sums, field placement, ranges, provenance. Schemas fix shape only.
Required information is absent from the source	Nullable field, null, and routing to review. Retries cannot recover it.
A failure reads as 'nothing found'	Return an error with category and retryability; keep valid empty results as successes.
Review finds nothing in code or reports the same agent produced	Independent instance with fresh context; per-file plus integration passes.
Latency-tolerant bulk workload	Batches (custom_id, no mid-request tools, worst-case SLA); never for blocking checks.
Too many agents or tools	Dynamic selection by complexity; scoped tools; one agent is fine for small tasks.
Subagent output lacks facts or sources	Pass complete structured findings in its prompt and require claim-source records back.
Context grows and answers get generic	Scratchpad, subagent delegation, /compact, structured summaries; keep numbers in a case-facts block.
Customer wants a human, or policy is silent	Escalate with a structured handoff. Do not use sentiment or confidence as the trigger.

Qualifier discipline

• **FIRST step**: the cheapest action that addresses the stated root cause (often descriptions, criteria, or a gate), not the most elaborate architecture.
• **SAFEST / MOST RELIABLE**: favors deterministic enforcement and least privilege over instructions.
• **BEST**: choose the option that satisfies all stated constraints (latency, cost, safety, ownership), not one that merely works.
• **Select TWO / THREE**: every selected option must be correct. Judge options independently, then count.

9. Cross-reference: official task statements to practice questions

Use this to find every item (across all papers) that assesses a task statement you want to review. IDs are Paper-Question (for example 3-14 is Paper 3, question 14).

Task	Topic	#	Questions
1.1	Agentic loops for autonomous task execution	10	1-3, 1-36, 2-2, 3-1, 3-5, 4-3, 4-7, 5-14, 5-15, 5-18
1.2	Coordinator-subagent orchestration	11	1-33, 1-34, 1-48, 3-2, 3-6, 3-17, 3-19, 4-30, 4-32, 5-17, 5-19
1.3	Subagent invocation, context passing, spawning	11	1-32, 1-35, 1-38, 3-3, 3-4, 3-16, 4-29, 4-31, 4-33, 5-16, 5-20
1.4	Multi-step workflows: enforcement and handoff	7	1-2, 1-6, 2-5, 4-1, 4-2, 4-4, 4-5
1.5	Agent SDK hooks: interception and normalization	14	1-4, 2-1, 2-3, 2-7, 2-46, 2-47, 3-48, 3-49, 3-50, 4-44, 4-45, 5-44, 5-45, 5-46
1.6	Task decomposition strategies	16	1-5, 1-18, 2-4, 2-6, 2-17, 2-32, 2-33, 2-34, 2-48, 3-33, 3-34, 3-35, 4-46, 5-30, 5-31, 5-32
1.7	Session state, resumption, and forking	11	1-1, 1-7, 1-37, 2-16, 2-18, 3-18, 4-6, 4-16, 5-1, 5-2, 5-21
2.1	Tool interfaces: descriptions and boundaries	11	1-8, 1-41, 2-50, 2-52, 3-9, 3-51, 4-9, 4-48, 5-25, 5-26, 5-50
2.2	Structured error responses for MCP tools	12	1-11, 1-39, 2-8, 2-35, 2-36, 3-10, 3-36, 4-47, 4-49, 5-22, 5-33, 5-49
2.3	Tool distribution and tool_choice	12	1-10, 1-49, 2-49, 2-51, 3-8, 3-11, 4-8, 4-34, 4-50, 5-24, 5-47, 5-48
2.4	MCP server integration (Claude Code and agents)	12	1-9, 1-40, 1-51, 2-9, 2-21, 3-7, 3-21, 3-37, 4-10, 4-36, 5-23, 5-34
2.5	Built-in tools: Read, Write, Edit, Bash, Grep, Glob	8	1-19, 1-50, 2-19, 2-20, 3-20, 3-22, 4-17, 4-35
3.1	CLAUDE.md hierarchy, scoping, modularity	9	1-23, 1-53, 2-22, 2-25, 3-23, 4-18, 4-21, 4-24, 5-3
3.2	Custom slash commands and skills	11	1-21, 1-25, 1-26, 1-55, 2-27, 3-24, 3-28, 4-20, 4-38, 5-4, 5-8
3.3	Path-specific rules	9	1-24, 2-24, 2-26, 3-25, 3-29, 4-22, 4-25, 5-6, 5-9
3.4	Plan mode vs direct execution	10	1-22, 1-52, 1-56, 2-28, 3-26, 4-19, 4-23, 4-37, 4-40, 5-7
3.5	Iterative refinement techniques	10	1-20, 1-54, 2-23, 2-29, 3-27, 3-39, 3-41, 4-39, 5-5, 5-10
3.6	Claude Code in CI/CD pipelines	11	2-37, 2-38, 2-39, 2-40, 3-38, 3-40, 3-42, 5-35, 5-36, 5-37, 5-38
4.1	Explicit criteria; reducing false positives	8	1-13, 1-14, 1-59, 2-10, 3-43, 3-45, 4-11, 5-41
4.2	Few-shot prompting	12	1-12, 1-27, 1-28, 1-45, 1-57, 1-58, 3-30, 4-26, 4-41, 4-42, 5-11, 5-12
4.3	Structured output via tool use and JSON schemas	10	2-42, 2-44, 2-56, 2-58, 3-47, 3-53, 4-51, 4-55, 4-57, 5-53
4.4	Validation, retry, feedback loops	11	2-41, 2-53, 2-55, 2-57, 3-46, 3-54, 4-52, 4-53, 4-58, 5-51, 5-55
4.5	Batch processing (Message Batches API)	11	2-43, 2-45, 2-54, 3-44, 3-52, 3-55, 3-56, 4-54, 4-56, 5-52, 5-54
4.6	Multi-instance and multi-pass review	8	1-42, 1-43, 1-44, 3-12, 5-27, 5-39, 5-40, 5-42
5.1	Context management across long interactions	8	1-15, 1-17, 1-31, 2-14, 2-15, 3-31, 4-14, 4-28
5.2	Escalation and ambiguity resolution	7	1-16, 2-11, 2-12, 2-13, 4-12, 4-13, 4-15
5.3	Error propagation across multi-agent systems	6	1-47, 3-14, 3-15, 5-28, 5-29, 5-59
5.4	Context in large codebase exploration	9	1-29, 1-30, 1-60, 2-30, 2-31, 3-32, 4-27, 4-43, 5-13
5.5	Human review workflows; confidence calibration	8	2-59, 2-60, 3-57, 3-60, 4-59, 5-43, 5-58, 5-60
5.6	Provenance and uncertainty in multi-source synthesis	7	1-46, 3-13, 3-58, 3-59, 4-60, 5-56, 5-57

10. Final week plan and exam-day checklist

Window	Focus
7-5 days before	Re-do your wrong-answer log. For each domain under about 70%, rebuild one small example: a loop with stop_reason, a hook with a gate, a schema with nullable fields, a path-scoped rule, a batch with custom_id.
4-3 days before	Skim section 7 and section 8 twice. Re-sit only the wrong items from Papers 1-3, after a 48-hour gap.
2 days before	Light review of out-of-scope list and your personal trap list. Sleep, logistics, and ID check (name must match registration).
Exam day	Read each stem fully; underline qualifiers and counts; answer, flag, move on; reserve time at the end; change answers only for a concrete reason.

Readiness checklist

[ ] I can state when to continue and when to stop an agentic loop, and what each stop_reason means.
[ ] I can choose between a prompt, a hook, and a gate for a given business rule, and justify it.
[ ] I can describe what subagents inherit (nothing) and what a coordinator must pass.
[ ] I can write a tool description that separates two similar tools and a structured error with category and retryability.
[ ] I can place any instruction at the right scope: user, project, directory, path-scoped rule, skill, or command.
[ ] I can choose plan mode or direct execution for a task and explain why.
[ ] I can design a schema that avoids fabrication (nullable, enums with 'other' and 'unclear') and add semantic validation.
[ ] I can decide when retries help and when they cannot, and bound them.
[ ] I can decide between synchronous and batch processing and compute a worst-case SLA, including retries.
[ ] I can set up independent and multi-pass review, and measure review quality by segment.
[ ] I can describe escalation triggers and why sentiment and self-reported confidence are poor ones.
[ ] I can explain provenance, conflict annotation, and coverage gaps in multi-source synthesis.

Good luck. These materials are a rehearsal, not a promise: the strongest preparation is building the systems the exam describes.